



PRÁTICAS DE SISTEMAS DIGITAIS - T02

Relatório de Práticas em Sistemas Digitais Primeira Unidade

{joaoccs,vonvictor,177013ot}@academico.ufs.br

João Carlos Cruz Santos, João Victor Carvalho Simões, Otávio
Augusto Batista Santos Filho

Orientador: Calebe Micael de Oliveira Conceição

Departamento de Computação – Universidade Federal de Sergipe (UFS) São
Cristóvão, SE – Brasil

Relatório da prática 10 – Lab 4

São Cristóvão, Sergipe 2025

SUMÁRIO

0.Resumo

1.Introdução

2.Entendendo o Circuito

3.Montagem do Circuito com Editor Esquemático

4.Prototipação do Circuito na Placa DE2

5.Montagem do Circuito em Verilog HD/ Prototipação da Versão em Verilog na DE2

6.Hora trabalho

6.1 Placas Xilinx

6.2 Placas Terasic

7.Questões Individuais

7.1 Questão 1

7.2 Questão 2

7.3 Questão 3

8.Referências Bibliográficas

0. Resumo

Este laboratório tem como objetivo proporcionar o conhecimento e a prática no uso da ferramenta Quartus II para a descrição e implementação de circuitos lógicos digitais, utilizando tanto o editor de esquemáticos quanto a linguagem de descrição de hardware Verilog em nível estrutural. As atividades desenvolvidas incluem a elaboração de projetos digitais, a realização de simulações funcionais, a síntese lógica e o mapeamento físico dos circuitos para a placa FPGA DE2, permitindo compreender o fluxo completo de desenvolvimento de sistemas digitais baseados em FPGA. Além disso, o experimento possibilita o entendimento do funcionamento geral de uma FPGA, bem como de sua arquitetura interna, destacando suas vantagens em termos de flexibilidade, desempenho e rapidez no desenvolvimento de protótipos digitais.

1. Introdução

O presente relatório tem como propósito descrever e analisar as atividades laboratoriais relacionadas ao desenvolvimento e implementação de um **circuito multiplexador 2 para 1** utilizando a plataforma de projeto digital **Quartus II**. Por

meio do experimento realizado, buscou-se aprofundar a compreensão dos princípios fundamentais que regem os circuitos digitais, adotando o funcionamento do multiplexador como elemento central para o estudo das técnicas de descrição, simulação e síntese de sistemas lógicos.

A abordagem empregada neste laboratório visa consolidar uma base teórica e prática no uso de ferramentas modernas de **descrição de circuitos lógicos**, contemplando tanto o editor de esquemáticos quanto a linguagem **Verilog**, em nível estrutural. Dessa forma, o estudante é introduzido aos conceitos essenciais de **linguagens de descrição de hardware (HDL)**, aos métodos de simulação funcional, à síntese lógica e ao processo de mapeamento físico para dispositivos programáveis.

Adicionalmente, a implementação do projeto na placa **FPGA DE2** permite compreender o funcionamento geral de uma **FPGA (Field Programmable Gate Array)**, explorando sua capacidade de reconfiguração, paralelismo e eficiência. Essa experiência prática contribui significativamente para o desenvolvimento das competências necessárias ao projeto e à análise de sistemas digitais modernos, estabelecendo uma conexão direta entre teoria, simulação e implementação física.

2. Entendendo o Circuito

O circuito em estudo, novamente, foi o Multiplexador 2 para 1, doravante chamado MUX2x1. Esse circuito possui três entradas - *sel*, *IN1*, *IN2* - quatro portas lógicas - duas AND, uma OR e uma NOT - e uma saída - *OUT*.

Sobre as funções de cada porta lógica, temos que, resumidamente: a) a porta AND gera saída 1 quando todas as entradas são 1; b) a porta OR gera saída 1 se pelo menos uma das entradas é 1; c) a porta NOT, também chamada de INVERSOR, inverte o sinal de entrada, ou seja, se a entrada é 1, ela gera 0, e se a entrada é 0, ela gera 1. Enfim, a saída *OUT* tem, como expressão lógica equivalente: $(In1 \& \sim Sel) \mid (In2 \& Sel)$.

Com essa expressão lógica equivalente, a tabela-verdade do MUX2x1 é a seguinte:

<i>Sel</i>	<i>In1</i>	<i>In2</i>	<i>Out</i>
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	0
0	1	1	1
0	1	0	1
0	0	1	0
0	0	0	0

Isso significa que a saída *OUT* é 1 se *IN1* tem valor e *Sel* tem valor 0. Por outro lado, *OUT* tem valor 0 se *IN2* e *Sel* têm, ambos, valor 1.

Essa estrutura lógica garante ao MUX2x1 a

função de selecionar um, entre dois valores de entrada, e encaminhá-lo para a saída, com base no valor de um selecionador, que cumpre a função de controle. Graças a isso, o MUX2x1 serve para:

- a) rotear dados em circuitos digitais;
- b) escolher entre duas fontes de informação;
- c) construção de unidades lógicas e aritméticas;
- d) implementar operações condicionais;
- e) selecionar sinais em sistemas embarcados e microprocessadores.

3. Montagem do Circuito com Editor Esquemático

Esta atividade iniciou-se com a criação de um novo projeto no Intel Quartus II, especificando-se o dispositivo correspondente à placa DE2. O Quartus II, enquanto Ambiente de Desenvolvimento Integrado, conduz todo o fluxo de síntese, otimização e *Place & Route*, mapeando o circuito descrito para os recursos físicos do FPGA — como CLBs, LUTs, flip-flops e redes de interconexão.

Em seguida, foi criado o arquivo **fig1.bdf**, no qual o circuito correspondente a um contador de 8 bits foi reproduzido graficamente. A implementação consistiu em replicar, por meio de portas lógicas,

registradores e conexões, a lógica funcional equivalente ao comportamento desejado: incrementar o valor binário armazenado a cada pulso de clock e reiniciá-lo quando o sinal de reset estivesse ativo.

Após a montagem, o circuito foi compilado com sucesso. O Quartus realizou a síntese e o mapeamento, produzindo uma netlist otimizada. Todo o processo foi concluído sem erros, resultando em utilização mínima dos recursos do FPGA — algo esperado, dado que contadores simples consomem poucas LUTs e flip-flops.

4. Prototipação do Circuito na Placa DE2

Na etapa seguinte, procedeu-se ao mapeamento dos sinais de entrada e saída do circuito para os pinos da DE2. O sinal de clock foi associado ao clock de 50 MHz da placa, o reset ao botão KEY0 (ativo-baixo), e o barramento de saída Q[7:0] aos LEDs vermelhos LEDR[7:0], após a síntese final, a configuração foi carregada com sucesso na FPGA. A placa passou a executar fisicamente o contador, incrementando seu valor binário a cada ciclo de clock. Como o clock nativo da DE2 é muito rápido para observação visual, utilizou-se um divisor de frequência pré-configurado no laboratório, reduzindo o sinal a aproximadamente 1 Hz.

Com isso, os LEDs passaram a exibir a contagem em binário, do valor 00000000 até 11111111, reiniciando após transbordo. A cada segundo, um novo padrão era ativado.

Por exemplo, nos primeiros oito segundos de operação, observou-se a seguinte sequência:

Tempo (s)	Saída Q (binário)
0	00000000
1	00000001
2	00000010
3	00000011
4	00000100
5	00000101
6	00000110
7	00000111

Comportamento totalmente coerente com a tabela-verdade e com a lógica proposta, com isto os relatórios de síntese apresentaram:

- a) utilização de 8 flip-flops;
- b) de 8 LUTs;
- c) inexistência de caminhos críticos relevantes;
- d) *timing slack* positivo (execução adequada a 50 MHz).

E todos os resultados foram compatíveis com a implementação esquemática

5. Montagem do Circuito em Verilog HDL

Para aprofundar o processo, reproduziu-se a mesma lógica utilizando Verilog HDL. Foi criado o arquivo **fig1.v**, contendo a descrição estrutural do contador:

```
module counter8bits ( input wire clk,
input wire RST_n, output reg [7:0] Q
);

always @(posedge clk or negedge RST_n)
begin if (!RST_n) begin
Q <= 8'b0;

end else begin

Q <= Q + 1'b1;

end

end

endmodule
```

A pinagem utilizada foi a seguinte:

```
clk = sw0;

RST_n = sw1;
Q[0]   = led0;
Q[1]   = led1;
Q[2]   = led2;
Q[3]   = led3;
Q[4]   = led4;
Q[5]   = led5;
Q[6]   = led6;
Q[7]   = led7;
```

Essa descrição representa exatamente a mesma lógica sintetizada no esquemático da Atividade 01, utilizando flip-flops disparados

por borda de subida e lógica combinacional simples para incrementar o registrador.

O Quartus II sintetizou o módulo sem erros, reconhecendo corretamente o registrador de 8 bits e o reset ativo-baixo.

Assim como anteriormente, foram utilizados apenas 8 flip-flops e uma quantidade mínima de LUTs — confirmando que o impacto estrutural da descrição textual equivalia ao do esquemático.

6. Hora trabalho

6..1 Placas Terasic

Foi realizada uma pesquisa nos sites da Terasic e da Xilinx com o objetivo de conhecer e comparar diferentes placas FPGA, concentrando a análise nas soluções oferecidas pela Terasic, por sua relevância para aplicações laboratoriais e profissionais. A investigação permitiu observar claramente a diferença entre placas voltadas ao ensino e prototipagem básica e aquelas destinadas a sistemas de alto desempenho.

Entre os modelos analisados, destacou-se a **Stratix V GX 100G**, considerada a mais avançada e de maior custo, com preço aproximado de US\$ 24.995,00. Essa placa é direcionada ao desenvolvimento e validação de sistemas de comunicação em altíssima velocidade, suportando taxas da ordem de 100 Gbit/s. Ela possui recursos como interfaces ópticas de alta performance, subsistemas avançados de memória DDR3 e QDRII, conectores de alta confiabilidade e suporte a

padrões modernos de comunicação, sendo indicada para aplicações profissionais e projetos industriais complexos.

Em contraste, a placa **Intel MAX 10** representa a opção mais acessível do catálogo, com preço aproximado de US\$ 49,95. Apesar do baixo custo, oferece um conjunto robusto de recursos voltados ao ensino e à prototipagem rápida, incluindo medição de consumo de energia, ajustes de tensão de I/O, memória flash integrada, ADC embutido e conectores compatíveis com Arduino. Essas características tornam a MAX 10 uma plataforma ideal para estudantes e desenvolvedores iniciantes.

A comparação entre os dois modelos evidencia o amplo espectro de aplicações das FPGAs, abrangendo desde ambientes educacionais até sistemas de altíssimo desempenho. Enquanto a Stratix V GX 100G atende às demandas de projetos avançados em comunicação de alta velocidade, a MAX 10 se destaca pelo excelente custo-benefício e simplicidade de uso, demonstrando a versatilidade e a diversidade do ecossistema FPGA.

6.2 Placas Xilinx

As placas de desenvolvimento da Xilinx, atualmente pertencente à AMD, apresentam ampla variação de preços, que pode ir de algumas centenas a vários milhares de reais. Essa diferença está diretamente relacionada à complexidade do FPGA empregado, à quantidade de recursos integrados, às interfaces disponíveis e aos periféricos embarcados.

De forma geral, as placas podem ser classificadas em três categorias principais:

- **Placas de baixo custo:** utilizam FPGAs mais simples, como as famílias Spartan-3A e Spartan-6. São voltadas ao ensino básico, estudantes e hobbystas, permitindo o desenvolvimento de projetos introdutórios em lógica digital e HDL. Seus preços variam aproximadamente entre R\$ 600 e R\$ 900.
- **Placas de médio custo:** utilizam dispositivos mais modernos, como a família Artix-7. Exemplos como Basys3 e Arty A7 oferecem bom equilíbrio entre custo e desempenho, sendo amplamente utilizadas no ensino de projetos digitais avançados, arquitetura de computadores e sistemas embarcados simples. Seus preços variam entre R\$ 1.000 e R\$ 3.000.
- **Placas de alto desempenho:** utilizam FPGAs avançados das famílias Kintex-7, Virtex-7 e Zynq UltraScale+, que podem integrar processadores ARM, transceivers multi-gigabit e interfaces de alta velocidade. São destinadas à indústria e à pesquisa, com aplicações em visão computacional, comunicações de alta velocidade, SDR e aceleração de algoritmos, apresentando custos a partir de R\$ 5.000.

Entre os principais recursos disponíveis nas placas Xilinx/AMD, destacam-se os **processadores ARM integrados**, que possibilitam a execução de sistemas operacionais embarcados e aplicações de alto nível combinadas com aceleração em hardware; os **transceivers de alta velocidade**, essenciais para comunicação de dados em taxas elevadas; as **interfaces de vídeo**, fundamentais para projetos de processamento de imagem; os **conectores FMC**, que permitem grande expansão

funcional; e as **memórias DDR/DDR4**, indispensáveis para aplicações intensivas em dados.

Esses recursos tornam as placas Xilinx extremamente versáteis, atendendo desde aplicações educacionais até sistemas industriais complexos. As áreas de aplicação mais relevantes incluem **visão computacional, sistemas embarcados inteligentes, processamento de sinais em tempo real, comunicação de alta velocidade e aceleração de algoritmos**, consolidando as FPGAs como uma das principais plataformas para projetos digitais modernos.

7. Questões Individuais

7.1). Quantos elementos lógicos foram utilizados na síntese usando a descrição gráfica?

Na síntese realizada por meio da descrição gráfica (editor de esquemáticos), foram utilizados **8 flip-flops e 8 LUTs (Look-Up Tables)**, compondo a estrutura lógica necessária para a implementação do circuito proposto.

7.2). Quantos elementos lógicos foram utilizados na síntese usando Verilog?

Na síntese realizada utilizando a linguagem de descrição de hardware **Verilog**, também foram utilizados **8 flip-flops e 8 LUTs**. Observa-se que a síntese resultou na **mesma estrutura lógica interna**, como era esperado, uma vez que ambas as descrições representam exatamente o mesmo circuito, apenas utilizando abordagens diferentes de modelagem.

7.3). Discuta sobre a diferença de implementação usando ambas as abordagens. Do ponto de vista pessoal, qual você acha mais agradável e por quê?

As duas abordagens de implementação diferem principalmente quanto à forma de desenvolvimento do projeto. A **descrição gráfica**, baseada em blocos, portas lógicas e conexões visuais, apresenta-se como uma ferramenta mais intuitiva, especialmente para circuitos pequenos e para usuários iniciantes. Essa abordagem facilita a compreensão do funcionamento do circuito, pois permite visualizar diretamente o fluxo dos sinais e as interligações entre os componentes.

Entretanto, à medida que a complexidade do circuito aumenta, a representação esquemática tende a se tornar visualmente poluída, exigindo uma estrutura gráfica extensa e complexa, o que pode dificultar a leitura, o entendimento e a manutenção do projeto. Em contrapartida, a descrição em **Verilog** permite alcançar o mesmo circuito final, porém oferece vantagens significativas em termos de **organização, versionamento, reutilização de módulos, escalabilidade e integração**, sendo especialmente adequada para projetos grandes e complexos.

Do ponto de vista pessoal, considero a implementação por meio da **descrição esquemática mais agradável**, devido à maior afinidade com representações visuais, à facilidade de edição e ao caráter mais didático e intuitivo. Além disso, essa abordagem torna o processo de desenvolvimento mais dinâmico e menos abstrato, especialmente para quem está habituado predominantemente ao trabalho com código textual. Contudo, reconheço que o **Verilog proporciona maior produtividade, flexibilidade e robustez**, sendo mais

apropriado para aplicações profissionais e projetos de maior porte.

8.Referências

[1] AMD. *FPGA & Adaptive SoC Devices – Product Line Overview*. 2025. Disponível em: <https://www.amd.com/en/products/adaptive-socs.html>. Acesso em: 22 nov. 2025.

[2] AMD. *Xilinx Development Boards – Official Product Catalog*. 2025. Disponível em: <https://www.amd.com/en/developer/xilinx.html>. Acesso em: 22 nov. 2025.

[3] DIGILENT. *FPGA Boards and Tools – Official Store*. 2025. Disponível em: <https://digilent.com/shop/>. Acesso em: 22 nov. 2025.

[4] QIU, R.; ZHANG, G. L.; DRECHSLER, R.; SCHLICHTMANN, U.; LI, B. *AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design*. In: ACM/IEEE International Symposium on Machine Learning for CAD – MLCAD '24, Snowbird (UT, USA), 9–11 set. 2024. New York: Association for Computing Machinery, 2024. p. 18. DOI: 10.1145/3670474.3685956.

[5] SCHULZ, R. (ed.). *Como Simular – Introdução à Verilog*. Campinas: Universidade Estadual de Campinas (UNICAMP), [s.d.]. Disponível em: <https://www.ic.unicamp.br/~rodolfo/Cursos/verilog/ComoSimular/>. Acesso em: 25 out. 2025.

[6] TOCCI, R. J.; WIDMER, N. S.; MOSS, G. L. *Sistemas Digitais: Princípios e Aplicações*. 12. ed. São Paulo: Pearson Education do Brasil, 2018.

[7] UYEMURA, J. P. *Sistemas Digitais: Uma Abordagem Integrada*. São Paulo: Thomson/Pioneira, 2002.

[8] ZHANG, Q.; SHANG, Y.; FANG, C.; GU, S.; ZHOU, J.; CHEN, Z. *TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models*. arXiv preprint, arXiv:2409.17561, 26 set. 2024. Disponível em: <https://arxiv.org/abs/2409.17561>. Acesso em: 26 out. 2025.